

01 | Day 2: Train It

When a neural network makes a mistake, how does it know what to change?

Retrieve: Open your submitted Day 1 consolidation. Do not edit it yet.

02 | Sample the Submitted Evidence

The instructor will sample one response:

- CHECK-D1-03 item 5: support/disconfirm nonlinearity;
- CHECK-D1-04 item 2: nonzero hidden activation;
- CHECK-D1-04 item 5: copied-parameter perturbation.

After discussion: annotate in a second color. Preserve the first state.

03 | Rebuild the Forward Trace

$$X \rightarrow Z_1 \rightarrow A_1 \rightarrow Z_2 \rightarrow p \rightarrow \hat{y}$$

For each stage, state one:

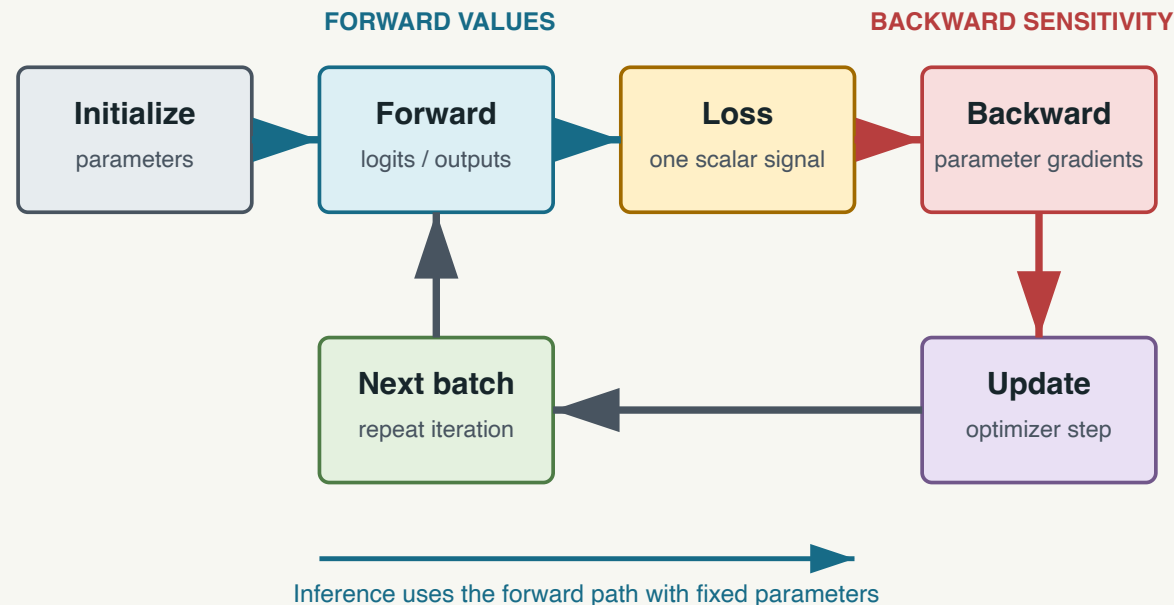
shape | range | invariant

Question: Which arrow changed a parameter on Day 1?

04 | The Forward Trace Becomes a Loop

One training iteration

Values move forward. Sensitivities move backward. The update happens after both.



05 | Batch, Iteration, Epoch

960 training examples | batch size 64

- batch: examples processed together;
- iteration: one update cycle;
- epoch: one pass through the training set.

Commit: How many updates occur in one epoch?

Training adds loss, backward, and update. Inference does not.

06 | Loss Gives Error a Scalar Form

One model can produce many mistakes across many examples.

Loss compresses the declared training objective into a scalar that gradients can act on.

Loss is not:

- the same as accuracy;
- a deployment cost by default;
- proof of generalization.

07 | ACT-D2-01: Rank Before Calculating

Targets: [1, 0, 1, 0]

Model	Probabilities for class 1
A	[.90, .10, .80, .20]
B	[.55, .45, .60, .40]
C	[.90, .10, .49, .20]
D	[.90, .10, .01, .20]

Commit: loss rank, accuracy, least-certain comparison.

08 | Equal Accuracy, Different Loss

Same decisions. Different optimization signal.

Bar length is probability assigned to the correct target class.

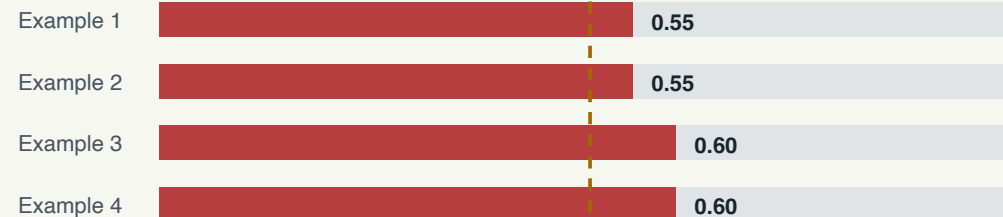
Model A



Accuracy 4/4

Mean BCE 0.164

Model B



Accuracy 4/4

Mean BCE 0.554

0.50 decision threshold

09 | Match Output and Loss

Task	Model emits	Introductory loss
regression	numeric output	MSE
binary class	one raw logit	BCE with logits
multiclass	one logit per class	cross-entropy

PyTorch 2.11: logits go directly to `BCEWithLogitsLoss` .

Sigmoid is for later interpretation or thresholding.

10 | Gradient Descent Uses Local Sensitivity

$$\theta_{next} = \theta - \eta \frac{\partial L}{\partial \theta}$$

- gradient sign: local slope; subtract it for descent;
- gradient magnitude + learning rate: update size.

Predict: crawl, smooth convergence, oscillatory convergence, or divergence?

11 | LAB-D2-01 Launch

Loss Landscapes and Learning-Rate Roulette | 40 min

Predict: BCE ranking and first three steps.

Experiment: finite loss and four fixed-budget paths.

Observe: position, loss, distance to optimum.

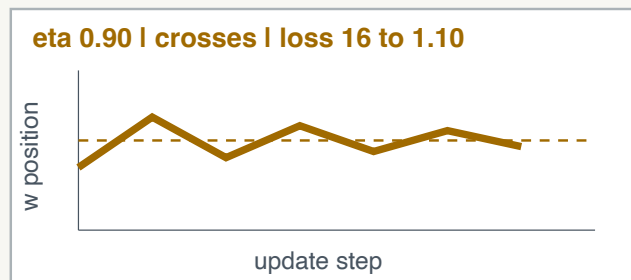
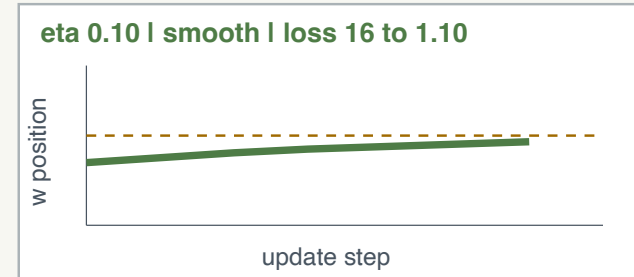
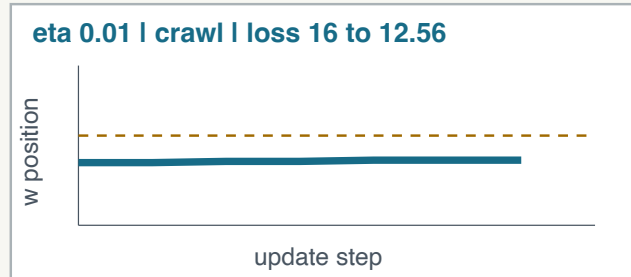
Diagnose: path from sign and update size.

```
courseware/day-2/labs/LAB-D2-01-loss-learning-rate.ipynb
```

12 | LAB-D2-01 Debrief: Read the Path

Learning rate changes the path, not the destination label

Lines show parameter position w . Dashed line is the optimum $w=3$. All panels use the same axes.



13 | A Derivative Is Local Sensitivity

Before notation, nudge one parameter:

$$\text{local sensitivity} \approx \frac{\text{small loss change}}{\text{small parameter change}}$$

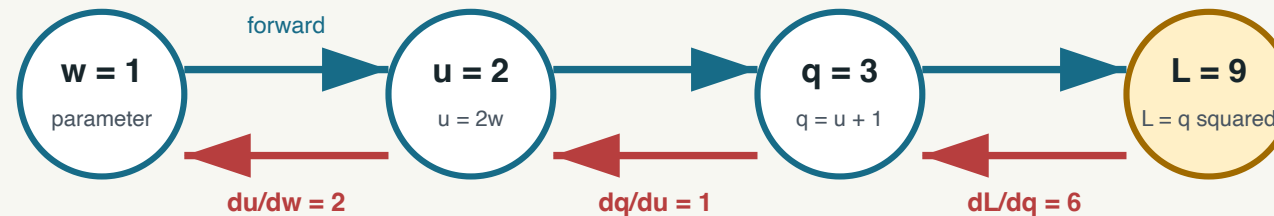
Predict sign before magnitude.

Backpropagation avoids separately perturbing every parameter.

14 | Backpropagation Reuses Local Work

One graph, two traces

Forward values above. Backward sensitivities below.



$$dL/dw = 6 \times 1 \times 2 = 12$$

upstream gradient x local derivative at each edge

15 | ACT-D2-02: Sign Before Arithmetic

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial w}$$

Upstream	Local	Product sign
positive	positive	?
positive	negative	?
negative	negative	?

Then: sketch the first three learning-rate steps.

16 | Gradient Checking Makes It Falsifiable

A backward pass makes a testable numerical claim

Centered finite differences check selected parameters at one point.

Parameter	Analytic	Numeric	Relative error	Interpretation
W2[0,0]	-0.171491679	-0.171491679	1.216e-10	agrees
b2[0]	-0.490282370	-0.490282370	7.912e-11	agrees
W1[0,0]	-0.033452121	-0.033452121	1.012e-09	agrees
sign bug	all layers	all disagree	1.000 max	opposite sign
missing factor	W1 / b1 only	W2 / b2 agree	0.648 hidden	localized factor

Agreement supports the derivative implementation here; it does not validate the data, objective, or model usefulness.

17 | LAB-D2-02 Launch

Backpropagation and Gradient Check | 45 min

Predict: selected gradient signs.

Build: forward cache and local derivatives.

Compare: analytic vs. finite difference.

Diagnose: sign bug or missing factor.

Verify: one small descent update lowers loss.

```
courseware/day-2/labs/LAB-D2-02-backprop-gradient-check.ipynb
```

18 | LAB-D2-02 Debrief

1. Why does one perturbation test an entire path?
2. What pattern suggests a missing local derivative?
3. When can epsilon mislead the check?
4. What remains unvalidated after gradients agree?

Must say: derivative correctness is not model usefulness.

19 | Midday Checks

CHECK-D2-01 | 5 min

Sequence the loop; repair logits/loss pairing.

CHECK-D2-02 | 5 min

Diagnose two learning-rate paths; trace one chain-rule calculation.

Submit before lunch through Day 2 Checks.

20 | ACT-D2-03: Add the Batch Dimension

Dense layer: 4 -> 8

Fill shapes for B = 1 , 32 , 1,024 :

X | W | b | Z = X @ W + b | A

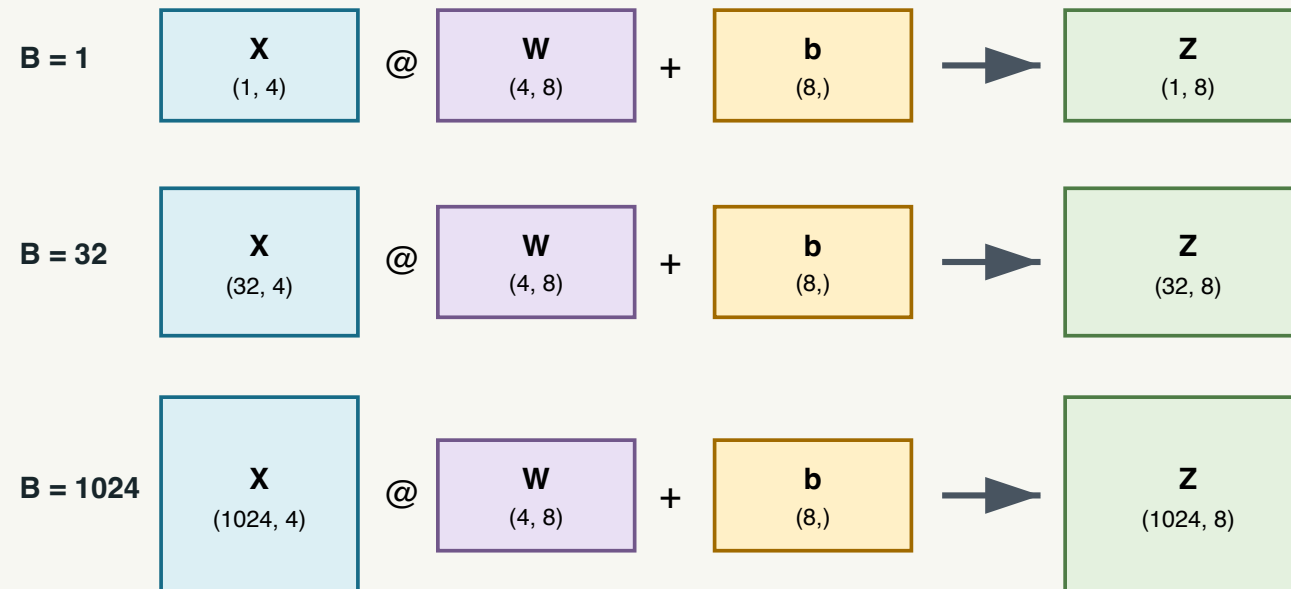
Then mark:

- contracted dimension;
- example dimension;
- trainable shapes that change.

21 | Same Function, More Rows

Batch size changes rows, not trainable shapes

The feature dimension 4 contracts. One bias vector broadcasts across examples.



22 | Vectorization Is a Computational Form

Same per-example function

Python loop -> batched matrix/tensor operation

Vectorized work can map efficiently to optimized kernels and parallel hardware.

It does **not** imply:

- GPU required;
- largest batch always fastest;
- larger batch always learns better.

23 | Scratch Training: Every Responsibility Is Visible

initialize -> forward/cache -> loss -> backward -> update -> mini-batch -> measure

At every stage ask:

- expected shape/range;
- finite value?
- current seed/split?
- evidence before continuing?

Decreasing loss is necessary evidence, not complete proof.

24 | LAB-D2-03 Launch

Train a Neural Network with NumPy | 70 min

Predict: initial loss, all shapes, early trend, boundary family.

Build: 2 → 8 → 1 forward/backward/update loop.

Observe: curves, gradient norms, validation, boundary.

Compare: one major change only.

`courseware/day-2/labs/LAB-D2-03-numpy-training.ipynb`

25 | Three Evidence Questions

Question	Evidence
Is the objective decreasing?	loss, gradient, update path
Can the model represent the boundary?	nonlinearity, decision region
Does behavior transfer beyond training?	validation evidence

Counterexample: loss can decrease while a no-activation model remains linear.

26 | LAB-D2-03 Debrief

Bring:

- one invariant that caught a defect;
- one curve supporting optimization progress;
- one boundary supporting or weakening representation;
- one validation observation;
- one result from your controlled change.

What should a framework automate next?

27 | PyTorch Automates the Mechanics You Exposed

The framework automates mechanics, not judgment

Every concise call maps to a responsibility already exposed in NumPy.

From scratch: NumPy	Automated: PyTorch 2.11
parameter arrays and shapes	registered <code>nn.Module</code> parameters
clear or replace gradient arrays	<code>optimizer.zero_grad(...)</code>
forward function plus cache	<code>logits = model(X_batch)</code>
stable binary loss	<code>BCEWithLogitsLoss(logits, targets)</code>
hand-written chain-rule gradients	<code>loss.backward()</code>
<code>theta = theta - eta * gradient</code>	<code>optimizer.step()</code>
fixed-parameter evaluation path	<code>model.eval() + inference_mode()</code>

Still yours: data, objective, evidence, diagnosis

28 | PyTorch 2.11 Training Order

```
model.train()
for X_batch, y_batch in train_loader:
    optimizer.zero_grad()
    logits = model(X_batch)
    loss = loss_fn(logits, y_batch)
    loss.backward()
    optimizer.step()
```

Probe: When do gradients exist? When do parameters change?

29 | Evaluation Mode Is Not Gradient Disabling

```
model.eval()  
with torch.inference_mode():  
    val_logits = model(X_val)
```

Control	What it changes
<code>model.eval()</code>	behavior of mode-sensitive modules
<code>torch.inference_mode()</code>	autograd recording and extra tracking in the block

They are orthogonal. Return to `model.train()` before training.

30 | LAB-D2-04 Launch

PyTorch Autograd: Break It and Fix It | 55 min

1. Build a valid logits-based baseline.
2. Record mode and gradient state.
3. Receive mystery evidence.
4. Diagnose before source reveal.
5. Repair one cause under matched conditions.

`courseware/day-2/labs/LAB-D2-04-pytorch-break-fix.ipynb`

31 | ACT-D2-04: Evidence Before Code

Submit before source access:

1. two observations;
2. two plausible causes;
3. cheapest discriminating check;
4. expected confirming result;
5. disconfirming result;
6. repair category.

A curve is not a unique fingerprint.

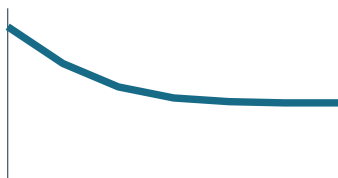
32 | Three Curves, More Than Three Causes

POST-GATE REVEAL

A symptom narrows possibilities; a check separates them

Each curve supports several hypotheses. Code stays hidden until a discriminating check is chosen.

Card A: stalled



Plausible:

- missing nonlinearity
- low effective update
- weak signal / wrong contract

Ask first:

- activation and boundary
- evidence under matched settings

Card B: unstable



Plausible:

- omitted gradient reset
- excessive learning rate
- invalid inputs / non-finite path

Ask first:

- gradient state before reset, after reset, after backward

Card C: inconsistent



Plausible:

- evaluation left in train mode
- stochastic data transform
- mutable state elsewhere

Ask first:

- fixed-batch repeats plus mode and grad state separately

33 | Repair Evidence + Exit Gate

LAB-D2-04 debrief:

symptom -> alternatives -> check -> fault -> repair -> before/after -> why

Final 15 minutes

- bounded modern-engineering transfer: 3 min;
- CHECK-D2-03 : 5 min;
- CHECK-D2-04 : 5 min;
- submit: 2 min.

Part C is due before Day 3 at 09:00.

34 | Training Is a Repeated Experiment

predict -> measure -> calculate sensitivity -> adjust -> repeat

The same scientific loop appears in larger training, post-training, evaluation, data, behavior, and efficiency work.

Boundary: no RL implementation, distributed training, or hardware internals today.

Day 3 bridge

Can a stronger model rescue invalid data and validation evidence?